

Parte 5 — Apêndices

Material de consulta: um glossário completo de todos os termos do projeto, uma tabela de solução de problemas reunindo os perrengues mais comuns, a ficha técnica do modelo, e caminhos para continuar aprendendo depois que o robô estiver pronto.

A. Glossário completo

Todos os termos do projeto, em ordem alfabética, em linguagem simples.

Atenção (self-attention) — O mecanismo do modelo que faz cada pedacinho de texto "olhar" para os outros e decidir em quais prestar atenção. É o coração do transformer.

Backpropagation — O cálculo automático de como cada parâmetro deve mudar para diminuir o erro durante o treino. No PyTorch, é a linha `loss.backward()`.

Bias — Um número extra somado numa camada, que dá mais flexibilidade ao ajuste.

block_size — O tamanho da "janela de contexto": quantos pedacinhos de texto o modelo consegue olhar de uma vez.

Cabeçalho (header) — A "etiqueta" no começo do arquivo `.bin`, com as dimensões do modelo, que o firmware lê antes dos pesos.

Cache KV — Uma memória que guarda as *keys* e *values* das posições já processadas, para o modelo não recalcular tudo a cada novo caractere.

Caractere (character) — No nosso projeto, o "pedacinho" de texto que o modelo processa. Trabalhamos com um modelo de nível de caractere (uma letra por vez).

Cross-entropy — A fórmula que mede o "erro" do modelo: o quanto ele se surpreendeu com a resposta certa. Erro alto = surpresa grande.

CUDA — A tecnologia que permite usar a placa de vídeo (GPU) para acelerar o treino.

Embedding — A transformação de um pedacinho de texto (um número) num conjunto de números (um vetor) que captura características dele.

ESP32-WROOM-32 — O microcontrolador do corpo do robô (navegação e sensores).

ESP32-S3 — O microcontrolador do cérebro (roda a LLM). Tem PSRAM e mais recursos.

FFN (rede feed-forward) — A parte do bloco onde cada pedacinho de texto é processado sozinho, depois de reunir contexto na atenção.

Firmware — O programa em C que roda dentro do microcontrolador.

Flash — A memória permanente do microcontrolador, onde ficam o firmware e os pesos.

float32 — Um formato de número com 32 bits (4 bytes). O formato "universal" em que exportamos os pesos.

Forward pass — A passagem dos dados pelo modelo, da entrada até a saída (os logits). É o "pensar" do modelo.

GND (terra) — O polo negativo do circuito. Regra de ouro: todos os GNDs unidos.

GPIO — Os pinos programáveis do ESP32 (General Purpose Input/Output).

HC-SR04 — O sensor ultrassônico que mede distância (o "olho" do robô).

Hiperparâmetros — As escolhas de configuração do modelo (`n_embd`, `n_layer`, etc.), definidas por você, não aprendidas no treino.

Inferência — Usar o modelo já treinado para gerar texto. O contrário de treinar.

L298N — A ponte H que controla os motores a partir dos sinais do ESP32.

- LayerNorm** — Uma operação que reequilibra os números do vetor para uma escala padrão, estabilizando o treino.
- Logits** — As "notas cruas" que o modelo dá para cada caractere possível, antes de virarem probabilidades.
- LLM (Large Language Model)** — Um modelo que aprendeu padrões de texto observando exemplos. O nosso é uma versão minúscula.
- Marcador** — Uma etiqueta curta de situação (`<obstacle>` , `<stuck>` , etc.) que o corpo envia ao cérebro.
- Máscara causal** — A regra que impede cada pedacinho de "ver o futuro": ele só olha para o que veio antes.
- Matmul (multiplicação de matrizes)** — A operação matemática central das camadas lineares.
- mmap** — Mapear um arquivo (ou partição) na memória, para lê-lo como se fosse um array, sem copiar.
- Modelo** — O programa que aprende padrões a partir de exemplos.
- n_embd** — O tamanho do vetor de cada pedacinho de texto (a dimensão do embedding).
- n_head** — O número de "cabeças" de atenção que olham o texto em paralelo.
- n_layer** — O número de blocos empilhados no modelo (sua profundidade).
- Parâmetros** — Os números ajustáveis que guardam o "conhecimento" do modelo.
- Partição** — Uma região da flash com um propósito específico (firmware, dados, etc.), definida no `partitions.csv` .
- Ponte H** — Um circuito que controla o sentido de giro de um motor. É o que o L298N é.
- Protoboard** — A placa de furos para montar circuitos sem solda, distribuindo VCC e GND.
- PSRAM** — Uma memória extra e maior do ESP32-S3, onde ficam os buffers de cálculo.
- PWM** — Um sinal que liga e desliga rápido para controlar velocidade (motores) ou ângulo (servo).
- PyTorch** — A biblioteca Python usada para construir e treinar o modelo.
- ReLU** — A "função de ativação" da FFN: mantém os positivos e zera os negativos.
- Residual (conexão residual)** — O "atalho" que soma a entrada de volta à saída de cada sub-camada, ajudando o treino.
- Servo (SG90)** — O pequeno motor que gira o sensor para varrer os lados.
- Softmax** — A operação que transforma notas em probabilidades que somam 100%.
- SRAM** — A memória rápida e pequena do microcontrolador, para os cálculos imediatos.
- Temperatura** — Um ajuste na geração: baixa = respostas mais "seguras"; alta = mais ousadas e variadas.
- Token** — Um pedacinho de texto. No nosso caso, um caractere.
- Tokenizer** — O tradutor entre texto e números (e vice-versa).
- Top-k** — Uma regra de geração que só considera os `k` caracteres mais prováveis no sorteio.
- Treino** — O processo de ajustar os parâmetros do modelo mostrando exemplos.
- UART / Serial** — A forma de comunicação por dois fios (TX e RX) entre placas, ou entre placa e computador.
- uv** — O gerenciador de pacotes Python usado no projeto.
- Vetor** — Uma lista de números. Um embedding é um vetor.
- Vocabulário** — A lista de todos os pedacinhos (caracteres) que o modelo conhece.
- Wokwi** — O simulador online gratuito de ESP32/Arduino.

B. Solução de problemas (troubleshooting)

Os perrengues mais comuns do projeto, reunidos.

Robô e eletrônica

Sintoma	Provável causa	Solução
Upload falha em "Connecting..."	ESP32 não entrou em modo de gravação	Segure o botão BOOT durante o "Connecting..."
ESP32 não liga com as pilhas	Jumper de 5V do L298N ausente	Coloque o jumper de 5V do L298N
Motores não giram	Jumpers ENA/ENB presentes	Remova os dois jumpers de ENA e ENB
Um motor gira ao contrário	Fios do motor invertidos	Troque os dois fios daquele motor nos bornes OUT
Sensor sempre lê o mesmo valor	TRIG/ECHO trocados ou mal ligados	Confira TRIG=GPIO32 e ECHO=GPIO33
Servo treme sem parar	Alimentação fraca	Garanta 5V estáveis e todos os GNDs unidos
Nada funciona / comportamento errático	GNDs não unidos	Una TODOS os GNDs (pilhas, L298N, ESP32, sensores)
LED_BUILTIN não compila	Não existe no ESP32	Use o GPIO2 diretamente

IA e treino

Sintoma	Provável causa	Solução
<code>torch.cuda.is_available()</code> dá False	Torch instalado sem CUDA	Configure o índice CUDA no <code>pyproject.toml</code> e rode <code>uv sync</code>
Mudanças com <code>uv pip install</code> somem	<code>uv run</code> re-sincroniza pelo lockfile	Declare tudo no <code>pyproject.toml</code> , não instale "por fora"
Erro cai muito devagar	Taxa de aprendizado baixa, ou modelo pequeno	Confirme <code>lr=1e-3</code> ; para qualidade, aumente o dataset
Frases geradas muito repetitivas	Top-k baixo ou temperatura baixa	Aumente levemente (ex.: <code>top_k=4</code> , <code>temperatura=0.75</code>)
Palavras "tortas" na geração	Modelo pequeno / dataset pequeno	Normal; melhora com mais dados e mais treino
Falta de espaço em disco no <code>.venv</code>	Cache do PyTorch é grande	Apague <code>.venv</code> ; o <code>uv sync</code> reinstala do cache

Embarque no ESP32-S3

Sintoma	Provável causa	Solução
Partição "model" não encontrada	Partition Scheme não é Custom	Ferramentas → Partition Scheme → Custom
Opção "Custom" não aparece	<code>partitions.csv</code> fora da pasta do sketch	Coloque-o na mesma pasta do <code>.ino</code> , de mesmo nome
Firmware grava em 0x10000, não 0x110000	Isso é normal	Firmware vai sempre em 0x10000; os pesos em 0x110000
Cabeçalho lê valores errados	Ordem dos pesos divergente	Confirme a mesma ordem no <code>export.py</code> e no firmware
PSRAM não detectada	PSRAM não configurada	Ferramentas → PSRAM → OPI PSRAM
Display não acende	Pinos SPI errados	Confira CLK=12, MOSI=11, CS=8, DC=9, RES=10
Interferência ao gravar firmware	Fio de UART conectado	Desconecte o fio de dados UART antes de gravar

C. Ficha técnica do modelo (referência rápida)

O modelo "nano-grump" na versão embarcada (Parte 4):

Item	Valor
Tipo	Transformer decoder-only, nível de caractere
Vocabulário	59 caracteres
Dimensão do embedding (n_embd)	64
Janela de contexto (block_size)	64
Número de blocos (n_layer)	4
Cabeças de atenção (n_head)	4
Parâmetros	~211 mil

Tamanho exportado (.bin)	~824 KB (float32)
Geração	top-k = 4, temperatura = 0.75
Marcadores	<start> , <explore> , <obstacle> , <turn_left> , <turn_right> , <backup> , <stuck> , <clear>

A Parte 3 usa uma versão didática ainda menor (~42 mil parâmetros) para o aprendizado. A arquitetura é a mesma; só mudam os números da configuração.

D. Mapa de pinos (referência rápida)

Robô (ESP32-WROOM-32):

Componente	Pino
HC-SR04 TRIG	GPIO32
HC-SR04 ECHO	GPIO33
Servo SG90	GPIO13
L298N ENA / IN1 / IN2	GPIO23 / 22 / 21
L298N IN3 / IN4 / ENB	GPIO19 / 18 / 5

Display OLED (ESP32-S3, SH1106 SPI):

Sinal	Pino
CLK	GPIO12
MOSI	GPIO11
CS	GPIO8
DC	GPIO9
RES	GPIO10

Comunicação entre as placas: TX do WROOM-32 → RX do S3, com os GNDs unidos.

E. Para onde ir depois

O robô está pronto — e agora? Alguns caminhos para continuar aprendendo e evoluindo o projeto:

- Melhorar o cérebro:** - Ampliar o dataset com mais frases e mais variadas (a maior alavanca de qualidade). - Refinar os marcadores mais fracos, dando mais exemplos de qualidade a eles. - Treinar por mais passos e comparar as perdas (loss) de treino e validação.
- Melhorar o corpo:** - Adicionar mais sensores (por exemplo, sensores laterais para não raspar em paredes). - Melhorar a lógica de exploração e o desvio de obstáculos. - Ajustar o "debounce" dos marcadores, para o cérebro não ser inundado de situações.
- Aprofundar o conhecimento:** - Estudar o `llama2.c` do Andrej Karpathy — a referência enxuta de inferência em C. - Ler sobre quantização (reduzir a precisão dos pesos para caber e acelerar). - Explorar modelos de nível de palavra ou de sub-palavra (BPE), além do de caractere.
- Compartilhar:** - Publique seu projeto (código e este tutorial) num repositório público. - Documente suas próprias descobertas e adaptações — ensinar é a melhor forma de consolidar o que se aprendeu.

Encerramento

Se você chegou até aqui e montou o projeto, parabéns de verdade. Você cruzou três mundos que raramente se encontram num tutorial só — eletrônica, robótica e inteligência artificial — e, mais importante, **entendeu cada um deles**.

Você não seguiu uma receita: você aprendeu a cozinhar. Agora o robô é seu, o conhecimento é seu, e os próximos projetos

também serão. 